

chrome_tab_organiser

v1.41 Editor Fix — Drag to Reorder Tabs

chrome_report_edit_v1.41.py

— a local web app to edit generated tab reports

— **fixed:** drag a tab up, down, or into another window — and it now actually moves

chrome_report_v1.4.py

— generate the HTML/JSON report this editor reads

Document suffix `_261004` — self-contained tools, Python standard library only.

Contents

- 1 What was wrong in v1.4
- 2 What v1.41 changes
- 3 How dragging works now
- 4 Files
- 5 Running the editor
- 6 Row positions (`data-pos`)
- 7 Saving
- 8 Testing
- 9 Troubleshooting
- 10 Relationship to the report generator

1 What was wrong in v1.4

Version 1.4 introduced the `⌵` drag grip, but the drag **never actually moved a tab** in a real browser. Three separate defects, all found by driving the real page with genuine native mouse-drag gestures:

1. **The drag never started.** The grip's `mousedown` handler called `preventDefault()`. In Chrome desktop, preventing the default action of a `mousedown` **suppresses drag initiation entirely** — so `dragstart` never fired. The grip still changed appearance on click, which made this look like it should work.
2. **Even when a drag did start, the session died instantly.** The `dragstart` handler mutated the DOM synchronously (adding the `ce-dragging` class to the row). A known Chrome bug aborts the drag session

when the DOM is touched inside `dragstart`: `dragend` fires immediately and no `dragover` / `drop` ever arrive.

3. **The commit depended on where you released.** The reorder only finalised on a `drop` event — but Chrome does not dispatch `drop` over the group chip, another row's grip, or a tab link (links are natively draggable themselves). Releasing anywhere except a bare table cell cancelled the drag.

The v1.4 headless test missed all three because it dispatched synthetic `DragEvent`s, which bypass Chrome's native drag-gesture machinery completely.

2 What v1.41 changes

#	Change	Fixes
1	The grip itself is the drag source : <code>draggable=true</code> is set when the row is built, long before any gesture. No <code>mousedown</code> handler, no <code>mousedown preventDefault</code> anywhere.	drag initiation
2	Rows are never draggable — text editing and link clicks in rows are untouched.	editing safety
3	All DOM changes in <code>dragstart</code> (dimming the row, arming the drag state) are deferred with <code>setTimeout(0)</code> .	session abort
4	The drag image is the whole row (<code>setDragImage</code>), not the tiny <code>⌵</code> glyph.	visibility
5	The reorder commits on <code>dragend</code> , which fires even when Chrome refuses to dispatch <code>drop</code> at the release point.	commit reliability
6	While a drag is live, buttons in the report are pointer-transparent, so <code>dragover</code> hit-testing falls through to the cells.	drop targeting

Because of change 5, you can now release the mouse **anywhere over a window's table** — over the group chip, over another row's grips, over a tab title link, or over a bare cell — and the move commits.

3 How dragging works now

Group	Tab	URL
 + × ChIP	Alpha	https://a.example/
← press and hold the grip, then drag; release anywhere over the table		

1. Press and hold the `⌵` **grip** in the Group cell and move the mouse.
2. The row dims; the drag ghost shows the whole row.
3. As the pointer crosses row midpoints, the row **moves live** to its slot; the target table gets a blue outline.
4. Release anywhere over the table. The move commits and `data-pos` is renumbered `1..N`.

Release point	Result
Above a row's midpoint	Row lands before that row
Below a row's midpoint	Row lands after that row
Past the last row	Row lands at the end
Over another window's table	Row moves into that window

Releasing **outside** any table (e.g. over the page margin) leaves the row where the last live move placed it — drag a little further onto a table and release again if you want a precise final position.

Starting a drag turns **sort groups alphabetically** off automatically, so your manual order is not reverted. Merely *clicking* the grip (press, no move, release) does **not** disturb the sort — only an actual drag does.

4 Files

File	Purpose
chrome_report_edit_v1.41.py	The editor. Run this.
chrome_report_edit_v1.41.exe	Standalone editor, no Python needed.
test_editor_js.py	Editor test suite — 33 checks, 0 failures.
chrome_report_edit_v1.4.py	Previous editor, kept for reference. Unchanged.
chrome_report_edit_v1.3.py ...	Older versions, unchanged.

The [_261004](#) guide set for v1.4 remains as history; this document supersedes it for v1.41.

5 Running the editor

From Python

```
python chrome_report_edit_v1.41.py
```

From the executable

Double-click [chrome_report_edit_v1.41.exe](#). On first run Windows may show a SmartScreen prompt — choose **More info** → **Run anyway**. The [.exe](#) manages the directory it lives in; point it elsewhere with [chrome_report_edit_v1.41.exe --dir "D:\path\to\reports"](#).

Options

Option	Meaning
--------	---------

<code>--dir PATH</code>	directory of reports to manage (default: the editor's own directory)
<code>--host HOST</code>	default <code>127.0.0.1</code>
<code>--port PORT</code>	default <code>8765</code>
<code>--no-browser</code>	do not open a browser window at startup

6 Row positions (`data-pos`)

Reports from `chrome_report_v1.4.py` tag each row with `data-pos="1"`, `"2"`, ... The editor rewrites them `1..N` after every **insert**, **delete**, **drag**, **group sort**, and **save**. Windows with no `data-pos` anywhere (older v1.3-era reports) are left that way — editing never silently adds the attribute.

7 Saving

Click **Save** or press `Enter` in the name box. Editor controls — the grip, the `+/×` buttons, the injected stylesheet, all `contenteditable` and drag-state attributes — are stripped automatically. A saved file is a clean report you can open in any browser.

8 Testing

```
python -m py_compile chrome_report_edit_v1.41.py test_editor_js.py
python test_editor_js.py
```

`test_editor_js.py` drives the real editor page in headless Chrome and asserts **33 behaviours**, including the regression tests added for this fix:

- grips carry `draggable=true` from the start; rows never do
- a stray drag from a tab cell or link is cancelled, order untouched
- drag down within a window reorders and renumbers `data-pos`
- drag across windows moves the tab; both windows renumbered
- **dragend alone commits without a drop event** and leaves clean state
- a mere grip click keeps the sort checked; an actual drag unchecks it
- delete and insert renumber
- save strips all editor chrome and keeps `data-pos`
- a legacy report without `data-pos` gains no attributes through any of it

The drag was additionally verified with **genuine native mouse-drag gestures** driven through the Chrome DevTools Protocol (continuous press → move → release over four release points: group chip, grip column, tab-title link, bare cell) — all four commit correctly. That is the machinery synthetic `DragEvent`s bypass, and it is what caught the v1.4 defects.

Requires Google Chrome; `Node.js` is **not** required.

9 Troubleshooting

The grip does nothing

The drag must start on the `⌵` button itself. Pressing on the group name, tab title, or URL edits that text instead.

I released and the row is not exactly where I wanted

The row sits where the last live move placed it. Release over the table, not over the page margin outside it; aim above or below a row's midpoint to choose the slot.

Rows went back to their old order

You probably had “sort groups alphabetically” checked. It turns itself off on the first actual drag; re-check it if you want the sorted view back.

The port is already in use

```
python chrome_report_edit_v1.41.py --port 8766 .
```

10 Relationship to the report generator

```
chrome_report_v1.4.py      --writes-->  chrome_tabs_Default_v1.4.html
                               |
                               opened in the editor
                               v
chrome_report_edit_v1.41.py <--saves--  any name you type
```

The editor reads and writes the same HTML the generator produces, so you can generate a report, tidy the order by dragging, and reopen the saved file whenever you like.